# DSC 412 - Scientific Computing Lab Report 2

**Neelay Kamat**
IMS23174

April 8, 2026

## Contents

**1 Introduction** **2**

**2 Steepest Descent with Newtons Method** **2**
2.1 Description . . . . . 2
2.2 Python Implementation . . . . . 2
2.3 Result Visualization . . . . . 3

**3 Modified Newton Method** **4**
3.1 Description . . . . . 4
3.2 Python Implementation . . . . . 4
3.3 Result Visualization . . . . . 5

**4 Conclusion** **5**

# 1 Introduction

This section presents implementations of optimization algorithms using numerical methods. The focus is on understanding convergence behaviour using gradient-based techniques and Newton-type methods.

# 2 Steepest Descent with Newtons Method

## 2.1 Description

This method computes the steepest descent direction and determines the optimal step size using Newton's method.

## 2.2 Python Implementation

Listing 1: Steepest Descent with Newton Line Search

```python
import numpy as np
import matplotlib.pyplot as plt

TOL = 1e-8
MAX_ITER = 20
ALPHA_TOL = 1e-10
MAX_ALPHA_ITER = 50

def f(x):
    x1, x2 = x
    return x1**2 + (x2**4)

def grad_f(x):
    x1, x2 = x
    return np.array([2*x1, 4*x2**3])

def hessian_f(x):
    x1, x2 = x
    return np.array([
        [2, 0],
        [0, 12*x2**2]
    ])

def compute_alpha_newton(x, p):
    alpha = 0.0
    for _ in range(MAX_ALPHA_ITER):
        x_new = x + alpha * p
        grad = grad_f(x_new)
        H = hessian_f(x_new)

        phi_prime = grad @ p
        phi_double_prime = p @ H @ p

        if abs(phi_prime) < ALPHA_TOL:
            break

        alpha = alpha - phi_prime / phi_double_prime
    return alpha
```

```python

x = np.array([4.0, 3.0])  # Starting point

points = [x.copy()]

for k in range(MAX_ITER):
    p = -grad_f(x)
    if np.linalg.norm(p) < TOL:
        break

    alpha = compute_alpha_newton(x, p)
    x = x + alpha * p
    points.append(x.copy())
```

## 2.3 Result Visualization

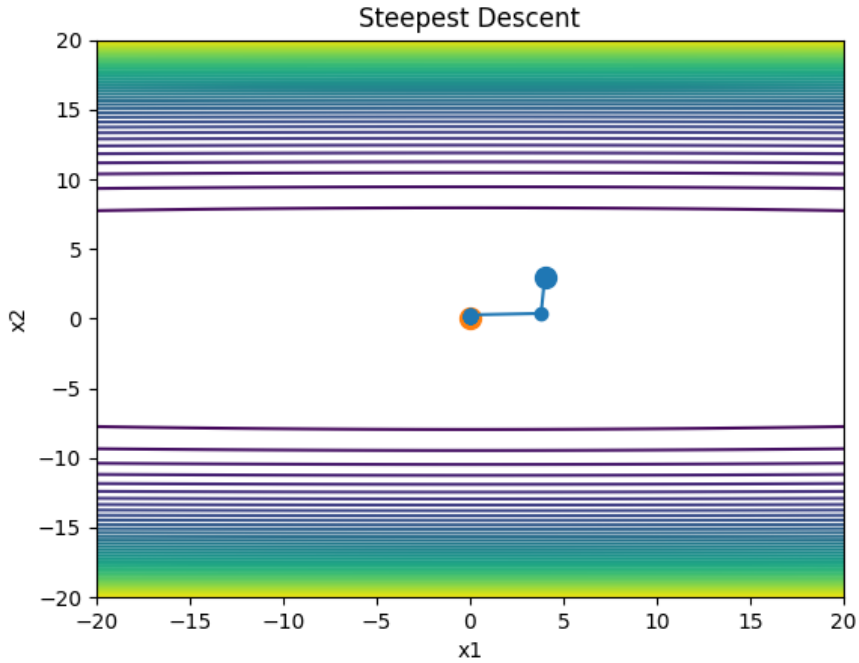


Figure 1: Steepest Descent with Newton Line Search starting from (4,3) for $f(x) = x_1^2 + x_2^4$

# 3 Modified Newton Method

## 3.1 Description

This method uses a modified Newton-Raphson approach to improve stability and convergence behaviour.

## 3.2 Python Implementation

Listing 2: Modified Newton Method (2D)

```python
import numpy as np
import matplotlib.pyplot as plt

MAX_ITER = 200
TOL = 1e-8
ALPHA_TOL = 1e-10
MAX_ALPHA_ITER = 50

def f(x):
    x1, x2 = x
    return x1**2 + (x2**4)

def grad_f(x):
    x1, x2 = x
    return np.array([2*x1, 4*x2**3])

def hessian_f(x):
    x1, x2 = x
    return np.array([
        [2, 0],
        [0, 12*x2**2]
    ])

def compute_alpha(x, p):
    alpha = 0.0
    for _ in range(MAX_ALPHA_ITER):
        x_new = x + alpha * p
        grad = grad_f(x_new)
        H = hessian_f(x_new)

        phi_prime = grad @ p
        phi_double_prime = p @ H @ p

        if abs(phi_prime) < ALPHA_TOL:
            break

        alpha = alpha - phi_prime / phi_double_prime
    return alpha

x = np.array([4.0, 3.0])  # Starting point

points = [x.copy()]

for k in range(MAX_ITER):
    grad = grad_f(x)
    if np.linalg.norm(grad) < TOL:
```

```
        break

    H = hessian_f(x)
    p = -np.linalg.solve(H, grad)

    alpha = compute_alpha(x, p)
    x = x + alpha * p
    points.append(x.copy())
```

### 3.3 Result Visualization

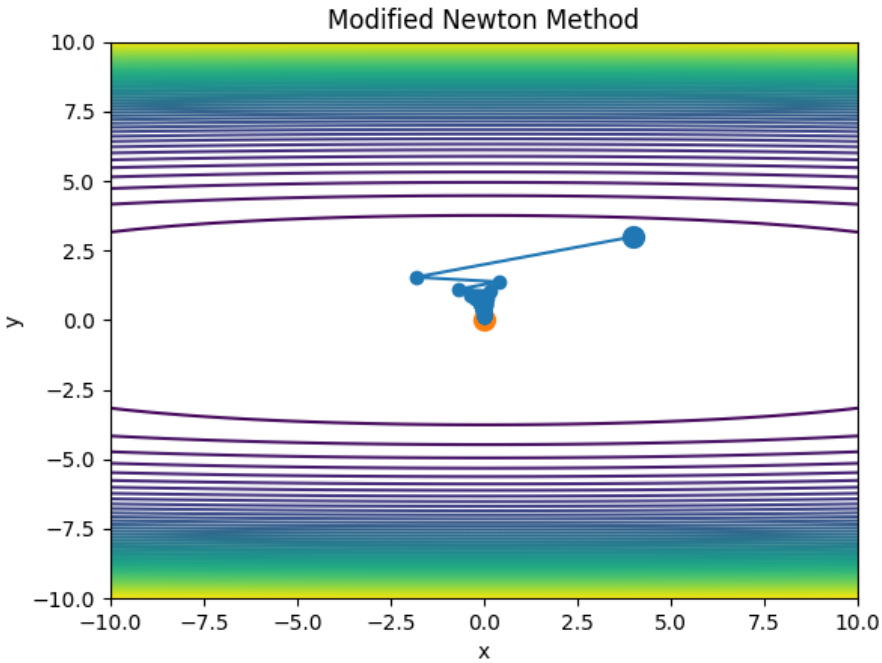


Figure 2: Modified Newton Method starting from (4,3) for $f(x) = x_1^2 + x_2^4$

## 4 Conclusion

The implementations demonstrate how Newton techniques can be integrated with both steepest descent and Newton methods to improve convergence behaviour. These approaches are especially useful for extending optimization methods to more general non-linear problems.